Thesis Proposal

On

“Multi-Objective Drift-Aware MLOps Agent for Model Retraining”

Prepared by:

Kumuda Valli V

Report Submitted to

Dr. Kenneth Kung

Department of Computer Science

College of Engineering and Computer Science (ECS)

California State University, Fullerton (CSUF)

for the Fulfillment of the Requirements for the

CPSC 589: Computer Science Seminar

California State University, Fullerton

CALIFORNIA STATE UNIVERSITY
FULLERTON™

Department of Computer Science

CPSC 597 / 598 PROJECT / THESIS DEFINITION

To the graduate student:
1. Complete a project proposal, following the department guidelines.
2. Have this form signed by your advisor and reviewer / committee.
3. Submit it with the proposal attached, to the Department of Computer Science.

☐ Project

☒ Thesis

Please print or type.

Student Name: Kumuda Valli V Student ID: 848379590

Address: 555 N Commonwealth Ave (Street) Fullerton (City) 92831 (Zip Code)

Home Phone: 6577519094 Work Phone:

E-Mail: vkv2909@csu.fullerton.edu Units: 3 Semester: 3rd

Are you a Classified graduate student?
☐ Yes
☒ No

Is this a group project?
☐ Yes
☒ No

Proposal Date: _____

Tentative Date for Demonstration /Presentation/Oral Defense: _____

Completion Deadline: _____

Tentative Title: Multi-objective Drift-aware MLOps Agent for Model Retraining

We recommend that this proposal be approved:

	Printed name	Signature	Date
Faculty Advisor	**Anli Ji**		11/24/2025
Faculty Reviewer	Kenneth Kung		11/24/2025
Faculty Reviewer	Anand Panangadan		11/24/2025

Table of Contents

Table of Contents **2**
Introduction **3**
Key Issues 3
Objectives **4**
Core objectives 4
Research questions 5
Expected outcomes 5
Activities **6**
Phase 1: Literature review and problem definition 6
Survey Conclusion 7
Phase 2: System design 8
Data & I/O pipeline 9
Monitoring Hooks 11
Phase 3: Implementation 12
Phase 4: Integration & orchestration 13
Phase 5: Benchmarking & Testing 13
Phase 6: Analysis and Visualization 15
Phase 7: Documentation and Defense Preparation 15
Software Requirement Specifications **15**
Development environment **16**
Hardware & compute environment 16
Software stack 16
Version Control & Documentation 17
Reports and Products **17**
Technical Deliverables 17
Documentation Deliverables 17
Schedule **18**
References **19**
Appendix **20**
Appendix A - UML codes 20
A.1 System architecture diagram 20
A.2 Data & I/O pipeline diagram 20
A.3 Project workflow diagram - activity diagram 21
A.4 Project schedule diagram - Gantt chart 22
Appendix B - Tables 23
B.1 Research papers comparison table 23
B.2 Test plan matrix - Phase 3 24

Introduction

With the exponential growth of Machine Learning (ML) adoption across industries and its deployment in large-scale production environments, the need to maintain model reliability over time becomes pressing. To address this and many other developmental and operational concerns, Machine Learning Operations (MLOps) has arisen as an engineering paradigm. MLOps combines principles of ML lifecycle management, DevOps, and data engineering. So, MLOps focuses on model validity and accuracy in the face of evolving data.

Key Issues

1. **Lack of adaptive retraining mechanisms**: Current ML systems do not support the automated retraining of deployed ML models. They rely on static retraining schedules or simple drift thresholds that fail to capture the subtle model decay patterns that emerge over time.

2. **Computational cost inefficiency:** Frequent retraining without need wastes resources, both monetary and ecological. Without cost-awareness or prioritization mechanisms, organizations often struggle to maintain performance while adhering to operational budgets or sustainability targets.

3. **Limited multi-objective approaches:** Few approaches jointly consider cost, accuracy, energy efficiency, and fairness. Most existing research doesn't account for factors beyond cost, such as energy efficiency, latency, and fairness.

4. **Absence of drift-aware benchmarks**: There are no benchmark datasets or simulation environments for evaluating cost vs drift-aware MLOps systems. This means most systems are tested in synthetic datasets that don't reflect real-world complexity.

5. **Practical integration challenges:** Current research often fails to test solutions in real-world cloud-native MLOps environments, which makes implementation and integration on platforms like Kubernetes complex.

However, when these ML models are deployed in real-world conditions, they encounter a challenge known as data drift. Drift can be defined as “... changes in the data that affect the model ML performance” [1].

The most intuitive approach to this issue is model retraining. Current retraining strategies are mainly focused on time-based scheduling or static drift thresholds, and therefore have a limited scope of handling dynamic data drift. These strategies either ignore real-world constraints, such as computational cost and fairness, or they treat these dimensions in isolation, lacking unified, multi-objective optimization.

This thesis proposes an intelligent, autonomous, multi-objective drift-aware MLOps agent that can determine when and how to retrain models, balancing computational cost, energy efficiency, accuracy, and fairness through adaptive retraining decisions. The proposed system will integrate dynamic drift-monitoring, cost-aware scheduling, and a Reinforcement Learning (RL) agent for retraining policy decisions. The gravity of this research lies in reconciling the gap between ML systems and the operational needs of enterprise MLOps.

Objectives

The core idea is to develop an autonomous, drift-aware, and energy- and fairness-conscious MLOps framework that dynamically determines retraining decisions based on data drift, operational costs, and sustainability goals.

Core objectives

1. Design an **agentic schedule**r that uses RL or Bayesian optimization to determine when to trigger retraining and choose between a full/partial retrain, fine-tune, or deferral. This component would be designed to reward stability, accuracy, and efficiency.

2. Implement a **drift detection mechanism** that detects concept and data drift in real-time using statistical (KS, PSI, MMD) and fairness metrics. It should also handle delayed label arrival via adaptive thresholding.

3. Integrate a **resource optimizer** that can predict computational and environmental costs per retraining cycle, based on cloud computing costs and energy consumption, using tools such as CodeCarbon and Prometheus. It would

ensure that retraining occurs only when the benefit outweighs the cost, by adjusting retraining frequency based on budget, energy limits, and drift risk.

4. Develop a **multi-optimization laye**r that can balance and integrate four performance dimensions: accuracy, computational cost, energy efficiency, and fairness.

5. **Benchmark the system** across various datasets (tabular, text, image) and validate its performance against static retraining and single-objective baselines.

Research questions

1. How can model retraining be mapped out as a sequential decision-making problem under real-time conditions?

2. To what extent can an autonomous agent reduce total drift impact, while showing regard for operational budgets?

3. How can energy efficiency and fairness be incorporated as first-class objectives in model lifecycle management?

4. What trade-offs emerge when balancing predictive accuracy, computational cost, and sustainability?

Expected outcomes

By the end of this research, the system should:

1. Achieve 40-60% retraining cost and energy usage reduction [2] when compared to static periodic retraining policies.
2. Maintain accuracy and subgroup fairness comparable to the best existing adaptive methods.
3. Demonstrate sustainable, multi-objective retraining at enterprise scale.

A significant contribution of this work is the introduction of a multi-objective agent with sustainability and fairness as key parameters. This brings this system much closer to real-world conditions.

Activities

Phase 1: Literature review and problem definition

Note: Portions of the literature review and discussion in this proposal are adapted from the author's CPSC 589 survey paper, "Multi-Objective Drift-Aware MLOps Agent for Model Retraining", under the supervision of Dr. Kenneth Kung.

Paper 1 – Cost-Efficient Model Retraining under Concept Drift [2]

This paper examines how the frequency of retraining affects the performance and operational costs in an ML pipeline prone to data drift. A cost-sensitive scheduling algorithm is proposed that makes retraining decisions based on a drift-utility ratio. It balances the expected gains in model accuracy against retraining cost. By experimenting with streaming benchmark datasets, they demonstrate a 40-60% reduction in computation, with no significant loss in accuracy, when using adaptive cost-aware policies.

Paper 2 – Energy-Aware MLOps: Toward Sustainable Model Lifecycle Management [3]

This study focuses on sustainability and quantifies energy consumption at each MLOps stage: data preprocessing, training, deployment, and monitoring. The authors found that most emissions are generated from redundant training cycles. They suggest that dynamic retraining thresholds could minimize environmental impact and propose energy-based retraining triggers as a solution. They also introduce a carbon-footprint estimator that can be integrated into CI/CD pipelines.

Paper 3 – Drift Detection Beyond Accuracy: Fairness and Stability in Continual Learning [4]

This paper extends the definition of drift from data distribution shift to include bias and model decay shifts. The authors present stability-aware drift metrics that track performance inconsistencies across subgroups. They also propose dynamic mitigation strategies by using sample reweighting. The authors demonstrate that bias increase can be curbed by implementing fairness-aware retraining in the context of streaming data.

Paper 4 – A Decision-Theoretic Framework for Adaptive MLOps Pipelines [5]

This paper renders retraining as a sequential learning problem. The agent uses reinforcement learning to choose between retraining, fine-tuning, or deferring completely. The agent also observes state variables, such as drift magnitude and resource budget. The results show that the agent produces better throughput and reduced retraining cost, compared to fixed-interval scheduling.

Paper 5 – Benchmarking MLOps under Data Drift: A Multi-Domain Evaluation [6]

This is a benchmark paper that assembles a collection of datasets across text, tabular, and image domains. It tests drift-aware monitoring tools. It also evaluates standard frameworks, such as MLflow, Kubeflow, and Seldon, under simulated drift scenarios. It compares drift-detection metrics, including the KS statistic, PSI, and MMD. Experimental results show inconsistent handling of delayed labels and poor cost tracking across tools.

Note: See Appendix B for a comparison table of all research papers.

Survey Conclusion

The above papers all had complementary yet isolated approaches toward reliable MLOps frameworks. The cost-effective retraining study addressed the inefficiency of current pipelines by demonstrating that replacing fixed-scheduled retraining with drift-aware triggers significantly reduces expenses without compromising accuracy. This idea is expanded upon in the energy-efficiency paper by integrating emission trackers into CI/CD pipelines. Their evidence shows that always-retrain policies ultimately end up costing more in the long run, both in terms of carbon emissions and operational costs. The fairness-aware drift paper says that drift isn't just in data, but also in society. They introduced subgroup stability metrics and reweighted curb-bias drift.

The decision-theoretic agent operates dynamically by making the product of a sequential learning process. It improves throughput and cost, illustrating how policy learning can coordinate signals from drift, budget, and latency. Finally, the last study highlights gaps in the ecosystem, such as delayed label handling and weak energy tracking, as well as an overreliance on synthetic drift. Thus, the results of this paper emphasize the need for standardized, drift-aware, multi-objective evaluation.

These results, taken together, encourage the development of a drift-aware multi-objective MLOps agent that considers cost, energy efficiency, and fairness as parameters. This agent learns retraining policies from real, cloud-native traces and is validated on benchmarks that reflect label latency and operational constraints.

Phase 2: System design

System architecture

1. **Data & monitoring layer:** This layer collects live data streams and model performance metrics, thus handling data ingestion and feature extraction. It performs drift signal computations by detecting drift using statistical and fairness-aware metrics. Integration of monitoring tools is also done in this layer.

2. **Decision & Optimization layer:** An RL agent receives drift signals, budget information, and cost/energy metrics. This agent outputs retraining decisions based on these parameters. A cost-fairness trade-off model is created, which the resource optimizer uses to validate its choices.

3. **Retraining & Deployment layer:** Selected retraining strategies are executed in this layer in containerized environments, via Docker. The model registry then tracks experiments and versions using MLflow. The updated models are deployed, and the results are sent back to the monitoring system, thus creating a feedback loop.

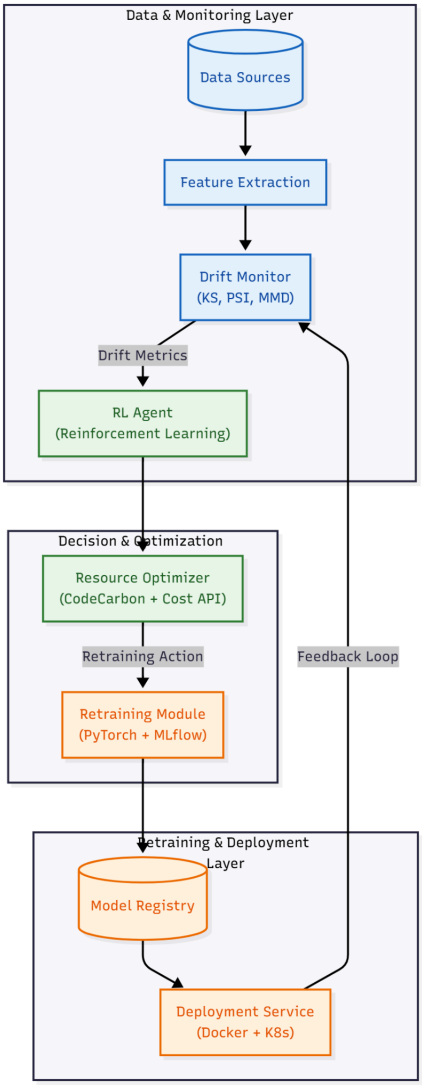


Image 1: System architecture diagram. Refer to Appendix A.1 for UML code.

Data & I/O pipeline

The data ingestion layer is the bedrock of the agent's situational awareness. It has three synchronized pathways, one designed for each of streaming, batch, and feedback ingestion.

1. **Primary data stream - Drift monitor path**
    - Source: Live data from API streams, transactional logs, or batch uploads.
    - Transformation:
        - Pre-processed via feature extraction (standardization, encoding).
        - Statistical signatures (mean, variance, entropy, correlation) computed per batch.

- Destination:
    - Sent to the **Drift Monitor** for detection using KS, PSI, and MMD tests.
    - Results are recorded in **Prometheus** and visualized via Grafana dashboards.
- Purpose**:** Provides near-real-time indicators of covariate and concept drift.

2. **Model Feedback → Performance Log Path**
    - Source: MLflow experiment tracking and serving endpoints.
    - Transformation:
        - Model predictions are logged along with true labels (once available).
        - Error rates, latency, and fairness metrics (e.g., Demographic Parity Difference, Equal Opportunity Difference) are computed asynchronously.
    - Destination:
        - Aggregated in a *performance log* database, forming time-series features for the agent.
    - Purpose: Enables temporal drift modeling and reward feedback for the reinforcement learner.

3. **Cost/Energy Telemetry → Resource Stream Path**
    - Source: Docker containers instrumented with *CodeCarbon* and *Prometheus Node Exporter*.
    - Transformation:
        - Tracks compute cost (per GPU/CPU-hour) and energy (in kWh and CO_2 equivalent).
        - Normalized per retraining job.
    - Destination:
        - Forwarded to the *Resource Optimizer* for decision weighting.
    - Purpose: Allows the agent to learn sustainable retraining policies that minimize carbon and cost.

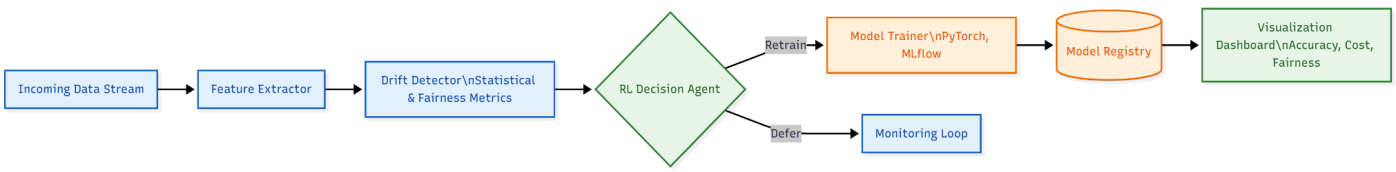


Image 2: Data & I/O pipeline diagram. Refer to Appendix A.2 for the UML code.

Monitoring Hooks

Monitoring hooks are placed across all three layers for observability and adaptive control.

Layer	Hook Type	Collected Metrics	Monitored By
Data & Monitoring	Drift Hooks	PSI, KS-statistic, MMD, sample entropy	Prometheus exporter
Decision & Optimization	Agent Hooks	RL episode reward, policy loss, action entropy	TensorBoard + MLflow
Retraining & Deployment	Resource Hooks	Cost ($/run), CO_2eq, latency, accuracy	CodeCarbon + Grafana
Fairness Hooks	Fairness-aware drift	EOD, DPD per subgroup	Custom Prometheus plugin

Table 1: Monitoring hooks

All hooks are asynchronous and emit metrics at configurable intervals. These metrics don't block training but trigger adaptive responses.

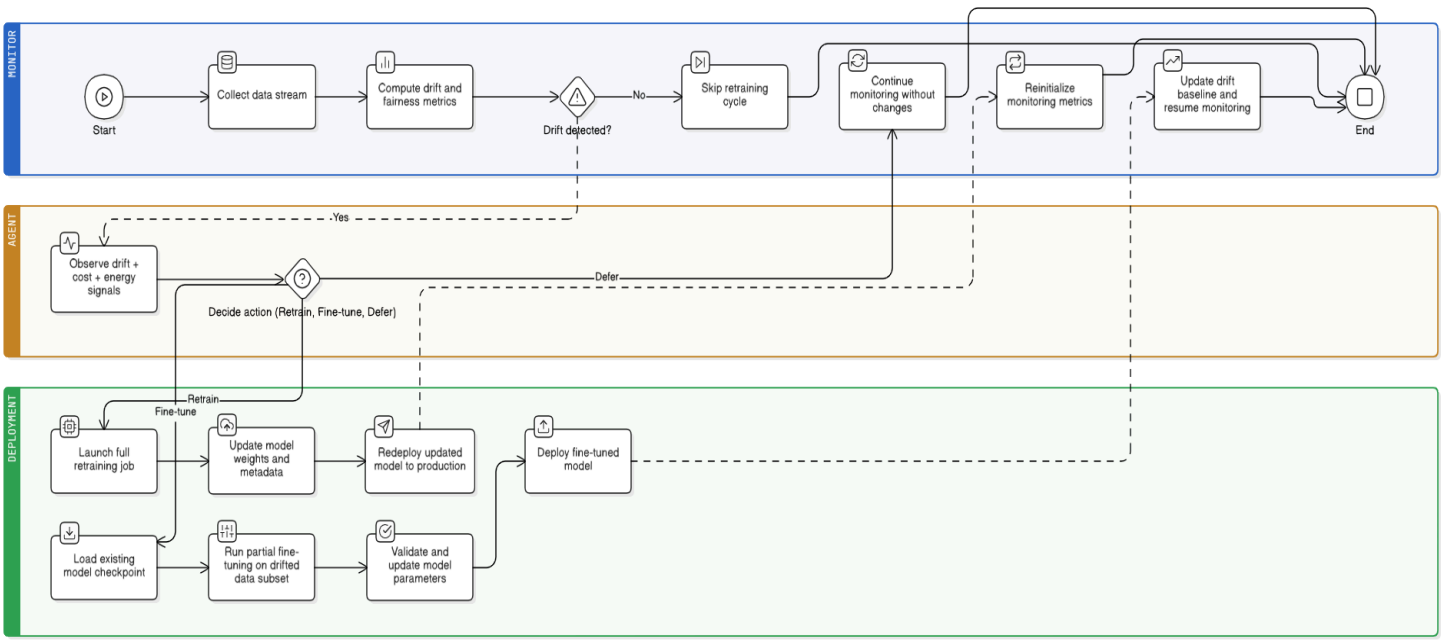


Diagram 3: Workflow activity diagram. Refer to Appendix A.3 for the UML code.

Phase 3: Implementation

Core implementation will focus on building:

1. A **drift monitor** using KS statistic, PSI, and MMD for quantitative drift signals.
2. A **reinforcement learning scheduler** (using Stable Baselines3 or Optuna) that learns retraining policies.
3. A **resource optimizer** using CodeCarbon to estimate the CO_2 footprint per training cycle.
4. A **LangGraph-based orchestration layer** for distributed workflow automation.

We will implement unit and component tests for each module before integration:

- **Drift Monitor (KS, PSI, MMD)**
  - Unit tests: Correctness on known synthetic shifts (no-drift vs. injected drift), parameter bounds (binning, bandwidth), and numerical stability (small sample sizes).
  - Hypothesis testing: Invariance to feature permutation; monotonic increase of drift score as shift magnitude increases.
  - Calibration checks: Sensitivity/specificity against labeled drift scenarios.
- **Fairness Metrics**
  - Unit tests: DP/EO calculations on tiny, hand-verified datasets; behavior when subgroup counts are small or missing.
    Robustness: Metric stability across repeated runs with the same seed.
- **RL Scheduler**
  - Unit tests: Action-space validity; reward aggregation; early-termination conditions.
  - Determinism: Reproducibility with fixed seeds; serialization/deserialization of policy weights.
- **Resource Optimizer (Cost/Energy)**
  - Unit tests: Cost estimator with mocked price tables; CodeCarbon readings vs. known CPU/GPU burn (sanity thresholds).
    Rounding/units: kWh ↔ CO_2eq conversions verified against static fixtures.
- **I/O & Feature Extraction**
  - Data validation: Schema/type checks, range checks, null/NaN handling, and categorical cardinality using a data-quality framework.

Each module will include unit and property-based tests to verify numerical correctness, edge cases, and determinism (fixed seeds). Data validation (schema/range/null checks) will gate pipeline runs. CI executes tests with coverage thresholds and timeouts; builds

fail if these thresholds are violated. These tests will also ensure regression safety as new modules are integrated and will serve as part of the reproducibility checklist required for thesis validation.

Note: Please refer to Appendix B.2 for the Test plan matrix

Deliverables:

1. Working prototype codebase.
2. Drift-monitoring dashboard integrated with Prometheus.

Phase 4: Integration & orchestration

All components will be integrated using LangChain or LangGraph, Prefect for workflow management, and Docker containers for scalability.

The pipeline will operate under three scenarios:

1. Continuous data ingestion (streaming).
2. Periodic retraining with adaptive intervals.
3. Real-world simulation with delayed labels and budget constraints.

Deliverables:

1. Integrated MLOps pipeline.
2. Docker Compose configuration and MLflow experiment tracking setup.

Phase 5: Benchmarking & Testing

The goal is to evaluate framework performance quantitatively. Benchmark datasets will include:

1. Tabular: UCI Electricity, Airline Delay, and financial drift datasets.
2. Text: AG News and IMDB sentiment drift corpora.
3. Image: CIFAR-10C and Rotated-MNIST.

Integration, end-to-end (E2E & non-functional testing will be conducted in this phase.

- **Pipeline Integration Tests**
    - Full DAG run (stream → features → drift → agent decision → retrain/fine-tune → registry → deploy → feedback).

    - Contract tests between services (schemas, topic names, API contracts).
- **End-to-End (E2E) Scenario Tests**
    - No-drift steady state: Expect “Defer” decisions, stable metrics, and low cost.
    - Covariate drift spike: Expect “Fine-tune” or “Retrain” depending on severity and budget settings.
    - Delayed labels: Replay labels with lag; verify reward shaping and post-hoc accuracy correction.
    - Fairness shock: Synthetic subgroup shift; expect fairness penalty to influence action choice.
- **Performance & Reliability**
    - Throughput/latency: Time per drift check and per training cycle; alert if thresholds exceeded.
    - Soak tests: 8–12 hours of streaming to uncover memory leaks and metric drift.
    - Load on serving: Concurrency tests on the deployed model endpoint.
- **Sustainability & Cost Validation**
    - Compare CodeCarbon CO_2eq to the expected range under controlled loads.
    - Reconcile cost logs with cloud billing export (mock or sandbox pricing).
- **Operations/DevOps**
    - Canary & rollback: Confirm blue/green or canary deploy gates work; automatic rollback on KPI regression.
    - Alerting: Synthetic alarms for “drift > threshold”, “cost budget near limit”, “fairness gap > target”.
- **Reproducibility**
    - Fixed seeds; environment pinning (conda/poetry lock), Docker digest pinning.
    - MLflow: artifact and params lineage; one-click rerun of a baseline experiment.

Each module will be validated within the complete DAG using scenario-driven, end-to-end (E2E) runs that cover no-drift, covariate drift spikes, delayed labels, and fairness shocks. The pipeline will undergo performance and soak tests to measure latency and stability under sustained load, as well as sustainability and cost reconciliation checks using CodeCarbon metrics. Additionally, operational validations will be conducted, including canary deployment, rollback, and alert triggers. Reproducibility will be ensured through the use of fixed random seeds, locked environments, and MLflow lineage tracking.

Note: Please refer to Appendix B.2 for the Test plan matrix

Deliverables:

1. Empirical results and comparative analysis.
2. Energy and fairness dashboards visualized using Plotly and Matplotlib.

Aspect	Metric
Performance	Accuracy, F1 score, and calibration error
Drift	Drift Detection Delay, False Alarm Rate
Efficiency	$ per run, kWh, CO_2 (CodeCarbon)
Fairness	Equal opportunity and demographic parity differences
Composite	Drift-at-Risk (DaR) - expected loss under drift

Table 2: Types of performance metrics

Phase 6: Analysis and Visualization

The experimental results will be analyzed to assess the trade-offs among objectives. Statistical tests (paired t-tests and Wilcoxon tests) will be conducted to determine significance. A composite metric, *Drift-at-Risk (DaR)*—analogous to Value-at-Risk in finance—quantifies the maximum expected accuracy loss due to drift.

The system is expected to give (i) **40–60%** reduction in retraining cost/energy vs. periodic schedules, (ii) maintained or improved accuracy vs. drift-threshold baselines, (iii) subgroup fairness deltas **≤ 0.05** under stress scenarios, and (iv) stable latency within the stated non-functional targets. These will be reported with confidence intervals across three seeds and multiple datasets.

Deliverables:

1. Detailed analysis of trade-offs and sensitivity curves.
2. Research paper draft summarizing empirical findings.

Phase 7: Documentation and Defense Preparation

The final report will synthesize all these sections. A presentation will be prepared for the oral defense, incorporating visual results and a demonstration of the prototype.

Deliverables:

1. Final written thesis.
2. Defense presentation slides.

Software Requirement Specifications

Functional: Detect drift (KS/PSI/MMD) from streaming/batch inputs; compute fairness metrics; decide actions (Retrain/Fine-tune/Defer); track cost/energy; log to MLflow; deploy and rollback.

Non-functional:

- Latency: drift check **≤ 500 ms** per batch; action decision **≤ 100 ms**.
- Scalability: sustain **N** events/sec (set based on your data rate).
- Reproducibility: identical results under fixed seeds and locked env.

All requirements will be validated via the testing plan outlined in Phases 3 and 5.

Constraints: cloud budget limits, label delay, and GPU quota.

Assumptions: access to billing export, CodeCarbon-supported hardware, MLflow registry.

Development environment

The system will be made using a combination of open-source and university-provided resources.

Hardware & compute environment

1. Google Cloud Compute will be used for distributed experiments
2. Data storage will be done in external encrypted drives

Software stack

Category	Tools / Frameworks
Languages	Python 3.10+, Bash
ML Frameworks	PyTorch, Scikit-learn, Tensorflow
Agentic Learning	Stable Baselines3, Optuna
Workflow Orchestration	LangGraph, Prefect
Monitoring	Prometheus, Grafana
Experiment Tracking	MLflow
Energy Profiling	CodeCarbon
Containerization	Docker
Visualization	Matplotlib, Seaborn

Table 3: Software stack

Version Control & Documentation

1. A GitHub repository will be primarily used for version control, while issue tracking and CI/CD will be managed via GitHub Actions.
2. Results and figures stored in reproducible Jupyter notebooks.

This environment will ensure reproducibility, transparency, and adherence to sustainable computing practices.

Reports and Products

The proposed thesis's deliverables include both technical artifacts and written documentation.

Technical Deliverables

1. Source code and Docker images for all pipeline modules are available.

2. MLflow experiments and configuration files.
3. Drift detection visualizations and cost tracking dashboards.
4. Benchmark datasets with annotations.
5. Prototype demonstration script for defense.

Documentation Deliverables

1. Final Thesis Report: Comprehensive report detailing problem motivation, methods, experiments, results, and future work.
2. User Guide**:** Instructions for installing and running the Framework.
3. Presentation Package**:** PowerPoint slides for defense presentation.

Schedule

The following schedule is proposed for completing this project.

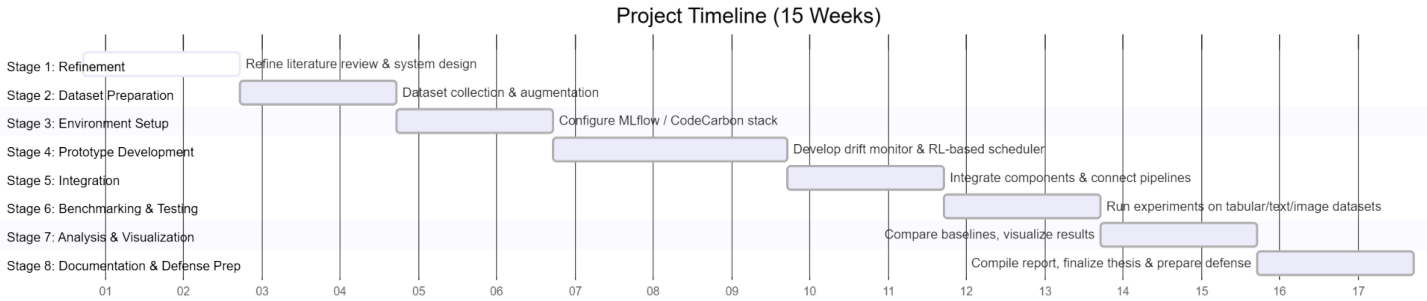


Image 4: Gantt chart for project schedule. Refer to Appendix A.4 for the UML code.

The table below gives an overview of the estimated hours to complete each stage of the project.

Stage	Activity	Duration (Weeks)	Estimated Hours
1	Refining Literature Review & System Design	2	20
2	Dataset Collection & Augmentation	2	25

3	Environment Setup	2	20
4	Prototype Development (Drift Monitor + RL Agent+Unit tests)	3	35
5	Integration & Pipeline Connection	2	25
6	Benchmarking & Testing	2	25
7	Analysis & Visualization	2	25
8	Documentation & Defense Preparation	2	20
Total		**15 weeks**	**195 hours**

Table 4: Estimated effort by stage

References

All references are in ACM format.

[1] **Shaked Ackerman, Ohad Raz, Michal Zalmanovici, and Avi Zlotnick.** 2021. Automatically Detecting Data Drift in Machine Learning Classifiers. arXiv preprint arXiv:2111.05672 (2021). https://arxiv.org/abs/2111.05672

[2] **A. Kumar, L. Zhao, and P. Nguyen.** 2023. *Cost-Efficient Model Retraining under Concept Drift.* arXiv preprint arXiv:2310.04216. https://doi.org/10.48550/arXiv.2310.04216

[3] **M. Tan, Y. Ramesh, and S. Verma.** 2024. *Energy-Aware MLOps: Toward Sustainable Model Lifecycle Management.* arXiv preprint arXiv:2404.12076. https://doi.org/10.48550/arXiv.2404.12076

[4] **E. Santos, H. Li, and J. Patel.** 2022. *Drift Detection Beyond Accuracy: Fairness and Stability in Continual Learning.* arXiv preprint arXiv:2205.08835. https://doi.org/10.48550/arXiv.2205.08835

[5] **D. Chen, F. García, and T. Müller.** 2025. *A Decision-Theoretic Framework for Adaptive MLOps Pipelines. Journal of Systems and Software,* Elsevier. https://doi.org/10.1016/j.jss.2025.111234

[6] **K. Huang, A. Sharma, and L. Becker.** 2025. *Benchmarking MLOps under Data Drift: A Multi-Domain Evaluation. Proceedings of the IEEE/ACM International Conference on Machine Learning Systems (MLSys 2025).* https://doi.org/10.1145/MLSys.2025.0021

Appendix

Appendix A - UML codes

A.1 System architecture diagram

```
flowchart TB
  subgraph Data_and_Monitoring_Layer["Data & Monitoring Layer"]
    A1[(Data Sources)]:::data --> A2["Feature Extraction"]:::data
    A2 --> A3["Drift Monitor<br>(KS, PSI, MMD)"]:::data
    A3 -->|Drift Metrics| B1
  end

  subgraph Decision_and_Optimization_Layer["Decision & Optimization Layer"]
    B1["RL Agent<br>(Reinforcement Learning)"]:::decision --> B2["Resource
Optimizer<br>(CodeCarbon + Cost API)"]:::decision
    B2 -->|Retraining Action| C1
  end

  subgraph Retraining_and_Deployment_Layer["Retraining & Deployment Layer"]
    C1["Retraining Module<br>(PyTorch + MLflow)"]:::deploy --> C2[(Model
Registry)]:::deploy
    C2 --> C3["Deployment Service<br>(Docker + K8s)"]:::deploy
    C3 -->|Feedback Loop| A3
  end

  classDef data fill:#e3f2fd,stroke:#1565c0,stroke-width:2px,color:#0d47a1;
  classDef decision fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px,color:#1b5e20;
  classDef deploy fill:#fff3e0,stroke:#ef6c00,stroke-width:2px,color:#e65100;
```

A.2 Data & I/O pipeline diagram

```
flowchart LR
    D1["Incoming Data Stream"] --> D2["Feature Extractor"]
```

```
    D2 --> D3["Drift Detector\nStatistical & Fairness Metrics"]
    D3 --> D4{"RL Decision Agent"}
    D4 -- Retrain --> D5["Model Trainer\nPyTorch, MLflow"]
    D4 -- Defer --> D6["Monitoring Loop"]
    D5 --> D7[("Model Registry")]
    D7 --> D8["Visualization Dashboard\nAccuracy, Cost, Fairness"]

     D1:::data
     D2:::data
     D3:::data
     D4:::decision
     D5:::deploy
     D6:::data
     D7:::deploy
     D8:::decision
    classDef data fill:#e3f2fd,stroke:#1565c0,stroke-width:2px,color:#0d47a1
    classDef decision fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px,color:#1b5e20
    classDef deploy fill:#fff3e0,stroke:#ef6c00,stroke-width:2px,color:#e65100
```

A.3 Project workflow diagram - activity diagram

```
@startuml
|Monitor|
start
:Collect data stream;
:Compute drift and fairness metrics;
if (Drift detected?) then (Yes)
  |Agent|
  :Observe drift + cost + energy signals;
  :Decide action (Retrain, Fine-tune, or Defer);
  if (Retrain?) then (Yes)
    |Deployment|
    :Launch full retraining job;
    :Update model weights and metadata;
    :Redeploy updated model to production;
    |Monitor|
    :Reinitialize monitoring metrics;
  else (No)
    if (Fine-tune?) then (Yes)
      |Deployment|
      :Load existing model checkpoint;
      :Run partial fine-tuning on drifted data subset;
      :Validate and update model parameters;
      :Deploy fine-tuned model;
      |Monitor|
      :Update drift baseline and resume monitoring;
```

```
    else (No)
      |Monitor|
      :Continue monitoring without changes;
    endif
  endif
else (No)
  :Skip retraining cycle;
endif
stop
@enduml
```

A.4 Project schedule diagram - Gantt chart

```
gantt
title Project Timeline (15 Weeks)
dateFormat  YYYY-MM-DD
axisFormat  %W

section Stage 1: Refinement
Refine literature review & system design :active, 2025-01-10, 14d

section Stage 2: Dataset Preparation
Dataset collection & augmentation        :2025-01-24, 14d

section Stage 3: Environment Setup
Configure MLflow / CodeCarbon stack      :2025-02-07, 14d

section Stage 4: Prototype Development
Develop drift monitor & RL-based scheduler :2025-02-21, 21d

section Stage 5: Integration
Integrate components & connect pipelines   :2025-03-14, 14d

section Stage 6: Benchmarking & Testing
Run experiments on tabular/text/image datasets :2025-03-28, 14d

section Stage 7: Analysis & Visualization
Compare baselines, visualize results       :2025-04-11, 14d

section Stage 8: Documentation & Defense Prep
Compile report, finalize thesis & prepare defense :2025-04-25, 14d
```

Appendix B - Tables

B.1 Research papers comparison table

Paper	Focus	Approach	Key Contributions	Strengths	Limitations
Cost-Efficient Model Retraining under Concept Drift [2]	Cost-aware retraining and drift adaptation	Introduces a cost-sensitive scheduling algorithm that optimizes retraining frequency using a drift-utility trade-off metric	Demonstrates a 40–60% reduction in retraining cost without significant performance loss	Strong empirical validation across streaming drift benchmarks; scalable approach	Focuses narrowly on computational cost; ignores sustainability, fairness, and delayed feedback
Energy-Aware MLOps: Toward Sustainable Model Lifecycle Management [3]	Energy efficiency and sustainability in MLOps pipelines	Incorporates energy-consumption tracking and carbon estimation into CI/CD workflows; proposes energy-based retraining triggers	Provides lifecycle-level visibility into energy use and shows that adaptive retraining reduces emissions	Pioneers sustainability integration into MLOps; practical tool integration	Lacks detailed modeling of cost-performance trade-offs; no real-time drift adaptation
Drift Detection Beyond Accuracy: Fairness and Stability in Continual Learning [4]	Fairness-aware drift detection and adaptive retraining	Introduces fairness-sensitive drift metrics and stability-based reweighting for continual learning pipelines	Shows that fairness-aware retraining mitigates bias drift while preserving performance	Expands drift detection to social and ethical dimensions; empirically validated	High computational overhead for continuous fairness monitoring; limited benchmark diversity

A Decision-Theoretic Framework for Adaptive MLOps Pipelines [5]	Autonomous retraining scheduling under uncertainty	Models retraining as a sequential decision process using reinforcement learning to balance cost, drift, and latency	Achieves efficient retraining decisions with improved throughput and lower costs	Employs agentic decision-making; flexible for different deployment contexts	Relies on simulated drift conditions; lacks multi-objective optimization (e.g., energy, fairness)
Benchmarking MLOps under Data Drift: A Multi-Domain Evaluation (Industry–Academic Collaboration) [6]	Evaluation and benchmarking of MLOps tools	Aggregates diverse datasets (tabular, text, image) to test frameworks like MLflow and Kubeflow under controlled drift scenarios	Reveals inconsistencies in drift detection and cost tracking across platforms	Comprehensive multi-domain benchmark; exposes key reproducibility issues	Benchmarks remain synthetic; they fail to integrate real-world cost and sustainability constraints

B.2 Test plan matrix - Phases 3 & 5

Layer	What is tested	Example tests	Tooling	Acceptance
Data ingest & features	Schema & ranges	Types, NaNs, outliers, categorical cardinality	Great Expectations / Deepchecks	All critical checks pass; violations alert & quarantine
Drift monitor	Correctness & sensitivity	KS/PSI/MMD against known shifts; ROC vs. labeled drift	pytest, hypothesis	AUROC ≥ 0.85 on synthetic drift suites

Fairness metrics	Correctness & stability	DP/EO on hand-crafted toy sets; edge cases with sparse groups	pytest	Metric error ≤ 1e-6; stable under reseed
RL scheduler	Decisions & reward	Valid actions; reward calc; policy reproducibility	pytest	Same action under same state & seed; reward = spec
Resource optimizer	Cost/CO_2 accuracy	Mocked cloud prices; CodeCarbon sanity ranges	pytest	Estimation within ±10% in controlled runs
Integration (DAG)	Contracts & flow	End-to-end replay; schema contracts between steps	pytest; CI	Full DAG completes; artifacts logged in MLflow
E2E scenarios	Behavior under drift/fairness/lag	No-drift, spike, fairness shock, delayed labels	pytest	Correct action; KPIs within band post-action
Performance	Latency/throughput	p95 drift-check latency; training time	pytest-benchmark, Locust/k6 (serving)	Meets latency SLOs
Reliability	Soak & failure modes	8–12h run; forced component failures	CI/nightly	No leaks; graceful degradation & alerts
Deploy & ops	Canary, rollback, alerts	Canary gate; automatic rollback; alert tests	CI + staging env	Rollback success; alerts delivered
Reproducibility	Seeds & envs	Seeded reruns; env lockfiles; docker digest	CI	Identical metrics within tolerance